

TAos User Manual

Version - Shows the user the current version of TAos they are running. This command changes output depending on the current version of the OS in use. Ran by inputting "version"

```
TAos> version
Current Version of TAOS: R2
```

Help - Used to show the user all possible commands able to be input at the current time. The help command provides a brief description of the command and aids in exact format to type into the terminal. Additionally, typing in help [commandname] provides a more indepth explanation of the command and its usage. Run by inputting "help", or for a more indepth tip, "help [commandname]".

```
TAos> help

Available Commands:
help [command] - Show this help message or show detailed help for given command
exit - Exit the OS; Asks the user for confirmation.
version - Displays the current version of the OS
getdate - Get the current date
setdate [date] - Set the current date
gettime - Get the current time.
settime [time] - Set the current time
```

```
TAos> help gettime

gettime: gettime
-----
Prints out the current system time. By default it is set to UTC time.
```

Get date - The getdate command provides the current date back to the user. It is important to note the command initially returns the current system date, but will instead

return whatever date was set using setdate if the OS has not been reset since then. The command can be used with the format “getdate”.

```
TAos> getdate
Current Date: 9/26/2025
TAos> setdate 09/24/2024
Date set to 9/24/2024
TAos> getdate
Current Date: 9/24/2024
```

Set date - The setdate command gives the user the ability to change the current date set in the system clock. This command overrides the previous date set in the system clock, and will remain as such until the system is reset or setdate is called again. This command is used with the format “setdate MM/DD/YYYY”. This command is set to UTC time by default. Possible errors include setting a month over 12, a day over 31, or a year over 9999.

```
TAos> setdate 12/02/2023
Date set to 12/2/2023
```

```
TAos> setdate 13/4/2000
Invalid month value. Please enter a value between 1 and 12.
```

```
TAos> setdate 12/02/99999
Invalid year value. Please enter a value between 0 and 9999.
```

Get time - The gettime command provides the current time back to the user. It is important to note the command initially returns the current system time, but will instead return whatever time was set using settime if the OS has not been reset since then. The command can be used with the format “gettime”. By default time is set in UTC.

```
TAos> gettime
Current Time: 19:13:23
```

Set time- The settime command gives the user the ability to change the current time set in the system clock. This command overrides the previous time set in the system clock, and will remain as such until the system is reset or settime is called again. This command is used with the format “settime hh:mm:ss”. This command is set to UTC

time by default. Possible errors include setting an hour over 23, a minute over 59, or a second over 59.

```
TAos> settime 13:20:24
Time set to 13:20:24
TAos> settime 24:20:22
Invalid hour value. Please enter a value between 0 and 23.
TAos> settime 13:20:99
Invalid second value. Please enter a value between 0 and 59.
```

Exit - Used to shutdown the current running operating system. This kills the terminal and resets many held values such as time and date. Can be called with “exit” followed by prompting “y” or “n” to confirm or cancel shutdown.

```
TAos> exit
Are you sure you want to exit?: (y)es or (n)o
no

Shutdown cancelled.
TAos> exit
Are you sure you want to exit?: (y)es or (n)o
yes

klogv: Starting system shutdown procedure...
klogv: Halting CPU...
```

****Deprecated****

Create pcb - Creates a new process control block. The user has to type create_pcb, name of the PCB, its class and priority to successfully run this command. Class 0 is a system pcb and 1 is a user PCB. The correct format is **create_pcb <name> <class:0|1> <priority:0-9>**

```
TAos> create_pcb
Usage: create_pcb <name> <class:0|1> <priority:0-9>
TAos> create_pcb proc 0 5
PCB 'proc' created
TAos> create_pcb son 0 t
Invalid priority.
TAos>
```

****Deprecated****

Delete pcb - Deletes an existing_process control block. The user has to type delete_pcb, and the name of the PCB to successfully run this command. The user cannot delete a system process.

```
TAos> delete_pcb proc
Cannot delete a system process
TAos> create_pcb son 1 6
PCB 'son' created
TAos> delete_pcb son
PCB 'son' deleted
TAos>
```

Block pcb - Changes the value of the 'state' field of an existing PCB from running to block. The user has to type "block_pcb name" to successfully run this command. Block_pcb is not able to block a pcb before it has been created. Block_pcb does not take additional inputs.

```
TAos> block_pcb proc
PCB 'proc' blocked
TAos>
TAos> block_pcb proc3
Process does not exist
```

Unblock pcb - Changes the value of the 'state' field of an existing PCB from block to ready. The user has to type "unblock_pcb name" to successfully run this command. Unblock_pcb can not unblock a nonexistent pcb. Unblock_pcb does not take additional inputs.

```
TAos> unblock_pcb proc
PCB 'proc' unblocked
TAos>
TAos> unblock_pcb proc3
Process does not exist
```

Suspend pcb - Changes the value of the 'state' field of an existing PCB from ready to suspend. The user has to type "suspend_pcb name" to successfully run this command. The user cannot suspend a system process. Suspend_pcb does not take additional inputs.

```
TAos> suspend_pcb proc
Cannot suspend a system process
TAos> suspend_pcb son
PCB 'son' suspended
TAos> 
```

Resume pcb - Changes the value of the 'state' field of an existing PCB from suspended to ready. The user has to type "resume_pcb name" to successfully run this command. Resume_pcb can not be called for pcb's that do not yet exist. Resume_pcb does not take additional inputs.

```
TAos> resume_pcb son
PCB 'son' resumed
TAos> 
```

```
TAos> resume_pcb proc2
Process does not exist
```

set pcb priority - Sets the value of the 'priority' field of an existing PCB to an integer between 0 to 9. The user has to type "set_pcb_priority name 0-9" The function will not allow setting the priority of a nonexistent pcb or setting the priority outside of the stated range.

```
TAos> set_pcb_priority son 9
PCB 'son' set to priority '9'
TAos> 
```

```
TAos> set_pcb_priority proc1 11
Invalid priority
```

```
TAos> set_pcb_priority proc5 1
Process does not exist
```


Show pcb - Show all the information regarding the PCB fields. The user has to type "show_pcb name" to successfully run this command. Show_pcb provides the user with pcb name, class, ready status, suspended status, and priority. The user is only able to show one pcb's details at once, you can not call multiple pcb's at once with this command.

```
TAos> show_pcb
Usage: show_pcb <name>
TAos> show_pcb son
PCB Information:
Name: son
Class: User
State: Ready
Suspended No
Priority: 9
```

```
TAos> show_pcb proc6
Process 'proc6' does not exist
TAos> show_pcb aaaaaaa aa a
Too many arguments. Usage: show_pcb <name>
```

Show ready pcb: Lists all the PCBs that are in the ready state. Shows the PCBs regardless of other current details as long as the PCB is in a ready state. Does not take any additional inputs. Used by calling "show_ready" in the terminal.

```
TAos> show_ready_pcb
Ready Processes:
Name: proc1, Class: User, State: Ready, Suspended: No, Priority: 4
Name: proc3, Class: User, State: Ready, Suspended: No, Priority: 4
Name: proc2, Class: User, State: Ready, Suspended: No, Priority: 5
TAos> 
```

```
TAos> show_ready
Ready Processes:
No processes in Ready state.
```

```
TAos> show_ready pr
The command 'show_ready' does not take any additional arguments.
```

Show blocked pcb: Lists all the PCBs that are in the blocked state. Shows the PCBs regardless of other current details as long as the PCB is in a blocked state. Does not take any additional inputs. Used by calling “show_blocked” in the terminal.

```
TAos> show_blocked
Blocked Processes:
Name: proc1, Class: System, State: Blocked, Suspended: No, Priority: 0
TAos> show_blocked
Blocked Processes:
No processes in Blocked state.
TAos> show_blocked 14
The command 'show_blocked' does not take any additional arguments.
```

Show all pcb: Lists all of the available PCBs currently created, regardless of current status and details. The command can be used with the format “show_all”. The command takes no additional inputs, and exists only to show current PCBs.

```
TAos> show_all
All Processes:
Name: proc3, Class: User, State: Ready, Suspended: No, Priority: 4
Name: proc2, Class: User, State: Ready, Suspended: Yes, Priority: 5
Name: proc1, Class: System, State: Blocked, Suspended: No, Priority: 1
TAos> show_all
All Processes:
No processes exist.
TAos> show_all 14
The command 'show all' does not take any additional arguments.
```

****Depreciated****

Yield: The yield command causes the command handler to yield to the CPU. It executes any processes in the queue, executing them. Yield takes no arguments, only being called with “yield” any additional statements will throw an error.

```
TAos> yield
Ready queue is empty - staying in command handler
Queue is empty
Returned to command handler
```

```
TAos> yield
dispatched proc1
exiting proc1
dispatched proc2
dispatched proc2
exiting proc2
dispatched proc3
dispatched proc3
dispatched proc3
exiting proc3
dispatched proc4
dispatched proc4
dispatched proc4
dispatched proc4
exiting proc4
dispatched proc5
dispatched proc5
dispatched proc5
dispatched proc5
dispatched proc5
exiting proc5
Queue is empty
Returned to command handler
TAos> []
```

```
TAos> yield r
The command 'yield' does not take any additional arguments.
```

****Deprecated****

Loadr3: Loads processes 1 through 5; (proc1-proc5) as R3 test processes. Will not loadr3 processes if has already been called and not yielded to. Loadr3 similarly throws an error if any additional arguments are provided. Loadr3 is called with "loadr3." The 5 processes are called in non-suspended ready state.


```
TAos> loadr3
Loaded proc1 successfully
Loaded proc2 successfully
Loaded proc3 successfully
Loaded proc4 successfully
Loaded proc5 successfully
Loaded 5 R3 test processes
```

```
TAos> loadr3
Process 'proc1' already exists, skipping
Process 'proc2' already exists, skipping
Process 'proc3' already exists, skipping
Process 'proc4' already exists, skipping
Process 'proc5' already exists, skipping
Loaded 5 R3 test processes
```

```
TAos> loadr3 ok
```

The command 'loadr3' does not take any additional arguments.

```
TAos> show_all
```

All Processes:

```
Name: proc1, Class: User, State: Ready, Suspended: No, Priority: 1
Name: proc2, Class: User, State: Ready, Suspended: No, Priority: 2
Name: proc3, Class: User, State: Ready, Suspended: No, Priority: 3
Name: proc4, Class: User, State: Ready, Suspended: No, Priority: 4
Name: proc5, Class: User, State: Ready, Suspended: No, Priority: 5
```

```
TAos> 
```

Loadr3 suspended: Works identically to loadr3 but loads the processes in the state of ready suspended. Called with "loadr3_suspended". Has similar errors to loadr3, as you can not call loadr3_suspended if previously called without yielding, and does not take additional parameters.

```
TAos> loadr3_suspended
loaded proc1 (suspended) successfully
loaded proc2 (suspended) successfully
loaded proc3 (suspended) successfully
loaded proc4 (suspended) successfully
loaded proc5 (suspended) successfully
loaded 5 R3 test processes (suspended)
TAos> loadr3_suspended
Process 'proc1' already exists, skipping
Process 'proc2' already exists, skipping
Process 'proc3' already exists, skipping
Process 'proc4' already exists, skipping
Process 'proc5' already exists, skipping
loaded 5 R3 test processes (suspended)
TAos> loadr3_suspended ok
The command 'loadr3 suspended' does not take any additional arguments.
```

```
TAos> show_all
All Processes:
Name: proc1, Class: User, State: Ready, Suspended: Yes, Priority: 1
Name: proc2, Class: User, State: Ready, Suspended: Yes, Priority: 2
Name: proc3, Class: User, State: Ready, Suspended: Yes, Priority: 3
Name: proc4, Class: User, State: Ready, Suspended: Yes, Priority: 4
Name: proc5, Class: User, State: Ready, Suspended: Yes, Priority: 5
TAos> 
```

Load process: Loads a single process of proc1 through proc5. The process is loaded as a ready non-suspended process. Called with the argument “load_process <name>” with name being either proc1,2,3,4, or 5.

```
TAos> load_process proc1
Loaded proc1 successfully
Loaded process 'proc1'
TAos> load_process proc2
Loaded proc2 successfully
Loaded process 'proc2'
TAos> load_process proc2
Process 'proc2' already exists
Loaded process 'proc2'
```

```
TAos> load_process proc
Unknown process name. Valid: proc1-proc5
```

Resume all: resume_all resumes all ready suspended processes. Resume_all takes no additional arguments and only works through calling "resume_all". The function has a singular function and doesn't work in any other manner.

```
TAos> loadr3_suspended
Loaded proc1 (suspended) successfully
Loaded proc2 (suspended) successfully
Loaded proc3 (suspended) successfully
Loaded proc4 (suspended) successfully
Loaded proc5 (suspended) successfully
Loaded 5 R3 test processes (suspended)
TAos> resume_all
Resumed 5 process(es)
Resumed all suspended processes
TAos> resume_all ok
The command 'resume_all' does not take any additional arguments.
TAos>
```

```
TAos> show_all
All Processes:
Name: proc1, Class: User, State: Ready, Suspended: Yes, Priority: 1
Name: proc2, Class: User, State: Ready, Suspended: Yes, Priority: 2
Name: proc3, Class: User, State: Ready, Suspended: Yes, Priority: 3
Name: proc4, Class: User, State: Ready, Suspended: Yes, Priority: 4
Name: proc5, Class: User, State: Ready, Suspended: Yes, Priority: 5
TAos>
```

```
TAos> resume_all
Resumed 5 process(es)
Resumed all suspended processes
TAos> show_all
All Processes:
Name: proc1, Class: User, State: Ready, Suspended: No, Priority: 1
Name: proc2, Class: User, State: Ready, Suspended: No, Priority: 2
Name: proc3, Class: User, State: Ready, Suspended: No, Priority: 3
Name: proc4, Class: User, State: Ready, Suspended: No, Priority: 4
Name: proc5, Class: User, State: Ready, Suspended: No, Priority: 5
TAos>
```


Alarm: creates an alarm process with a custom time and message made by the user. The alarm command takes 2 arguments, the time(UTC) in the HH:MM:SS format and the message separated by quotation marks(" "). Both Alarms and Loadr3 processes were accounted for in the dispatching operations.

Command Format: alarm HH:MM:SS "Message to display"

```
IDLE PROCESS EXECUTING.
TAos> get_time
Current Time: 03:56:33
IDLE PROCESS EXECUTING.
TAos> alarm 03:57:20 "This message will display at the right time"
Created alarm process for time 03:57:20
Alarm set successfully
TAos> show_all
All Processes:
Name: alarm_1, Class: User, State: Ready, Suspended: No, Priority: 3
Name: sys_idle, Class: System, State: Ready, Suspended: No, Priority: 9
TAos> get_time
Current Time: 03:57:00
TAos> get_time
Current Time: 03:57:10
TAos>
TAos>
TAos> get_time
Current Time: 03:57:18
TAos>
ALARM: This message will display at the right time
TAos> 
```

Allocmem: Allocates heap memory by calling "allocmem <size>" and prints the hexadecimal address of newly allocated block. Only allows size values within 50k and positive values. Will not work if system has run out of free memory to allocate.

```
TAos> allocmem 500
Allocated 500 bytes at address: 0x0011794E
IDLE PROCESS EXECUTING.
TAos> allocmem -500
Error: Size must be positive
IDLE PROCESS EXECUTING.
TAos> allocmem 1300000
Error: Could not allocate 1300000 bytes
IDLE PROCESS EXECUTING.
TAos> 
```

Freemem: Frees heap memory by calling “freemem <address>” and prints either a success message or error message depending on if memory could be freed. Shows inputted address attached to memory block. Will not work if system does not have allocated memory to free.

```
TAos> allocmem 10
Allocated 10 bytes at address: 0x00117BBD
IDLE PROCESS EXECUTING.
TAos> freemem 10
Error: Could not free memory at address: 0x00000010
IDLE PROCESS EXECUTING.
TAos> freemem 0x00117BBD
Successfully freed memory at address: 0x00117BBD
IDLE PROCESS EXECUTING.
```

Showalloc: Shows all allocated blocks of memory and their corresponding addresses. Called using “showalloc” and does not take additional arguments. Showalloc prints no allocated blocks if no blocks of memory are currently allocated.

```
TAos> showalloc
Allocated Memory Blocks:
Address: 0x00117014, Size: 24 bytes
Address: 0x00117040, Size: 64 bytes
Address: 0x00117094, Size: 1024 bytes
Address: 0x001174A8, Size: 64 bytes
Address: 0x001174FC, Size: 1024 bytes
Address: 0x00117910, Size: 8 bytes
Address: 0x0011792C, Size: 10 bytes
IDLE PROCESS EXECUTING.
TAos> allocmem 30
Allocated 30 bytes at address: 0x0011796A
IDLE PROCESS EXECUTING.
TAos> showalloc
Allocated Memory Blocks:
Address: 0x00117014, Size: 24 bytes
Address: 0x00117040, Size: 64 bytes
Address: 0x00117094, Size: 1024 bytes
Address: 0x001174A8, Size: 64 bytes
Address: 0x001174FC, Size: 1024 bytes
Address: 0x00117910, Size: 8 bytes
Address: 0x0011792C, Size: 10 bytes
Address: 0x0011794A, Size: 12 bytes
Address: 0x0011796A, Size: 30 bytes
Address: 0x0011799C, Size: 10 bytes
IDLE PROCESS EXECUTING.
```


Showfree: Shows all free memory blocks and their corresponding addresses. Called using “showfree” and does not take additional arguments. Showfree prints no free memory blocks if no blocks of memory are currently free.

```
TAos> showfree
Free Memory Blocks:
Address: 0x001179D7, Size: 47481 bytes
IDLE PROCESS EXECUTING.
TAos> freemem 0x0011796A
Successfully freed memory at address: 0x0011796A
IDLE PROCESS EXECUTING.
TAos> showfree
Free Memory Blocks:
Address: 0x0011792D, Size: 9 bytes
Address: 0x0011796A, Size: 30 bytes
Address: 0x001179FE, Size: 47442 bytes
IDLE PROCESS EXECUTING.
```